

EXERCISE NO.6

Using existing (pre-defined) Functional Interfaces in Java write:

1. Lambda expression to sort a string array in alphabetical order.

Code:

```
import java.util.Arrays;
public class Main {
    public static void main(String[] args) {
        String[] names = {"Kajal", "Rahul", "Kiyan", "Aishwarya", "Bhavesk"};

        // Arrays.sort() uses the Comparator to decide the order.
        Arrays.sort(names, (first, second) -> first.compareTo(second));
        System.out.println(Arrays.toString(names));
    }
}
```

2. Lambda expression to find the largest number in an integer array.

Code:

```
import java.util.function.Function;
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 45, 23, 89, 12, 67};
        // Function:
        // takes int[] as input
        // returns Integer as output
        Function<int[], Integer> largest = arr -> {
            int max = arr[0];
            for (int i = 1; i < arr.length; i++) {
                if (arr[i] > max) {
                    max = arr[i];
                }
            }
            return max;
        };
        // Calling lambda using apply() which return result
        int result = largest.apply(numbers);
        System.out.println("Largest = " + result);
    }
}
```

3. Lambda expression to find the smallest number in an integer array.

Code:

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 45, 23, 89, 12, 67};

        // Function takes int[] and returns Integer
        Function<int[], Integer> smallest = arr -> {
            int min = arr[0];
            for (int i = 1; i < arr.length; i++) {
                if (arr[i] < min) {
                    min = arr[i];
                }
            }
            return min;
        };

        int result = smallest.apply(numbers);
        System.out.println("Smallest = " + result);
    }
}
```

4. Lambda expression to generate a 3 digit random number.

Code:

```
import java.util.Random;
import java.util.function.Supplier;

public class Main {
    public static void main(String[] args) {
        Random random = new Random();
        // Supplier
        // takes NO input ,returns Integer
        Supplier<Integer> threeDigitNumber = () -> {

            // nextInt(900) gives numbers from 0 to 899. Adding 100 gives 100 to 999.

            return random.nextInt(900) + 100;
        };
        // Supplier uses get() to return result
        int number = threeDigitNumber.get();
        System.out.println("Random 3 digit number = " + number);
    }
}
```

5. Lambda expression that takes an integer array and returns the reverse integer array.

Code:

```
import java.util.Arrays;
import java.util.function.Function;
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // Function: // input -> int[] , output -> int[]
        Function<int[], int[]> reverse = arr -> {
            int[] reversed = new int[arr.length];
            for (int i = 0; i < arr.length; i++) {
                reversed[i] = arr[arr.length - 1 - i];
            }
            return reversed;
        };

        // apply() executes the lambda which return result
        int[] result = reverse.apply(numbers);
        System.out.println(Arrays.toString(result));
    }
}
```

6. Lambda expression to print the current date

Code:

```
import java.time.LocalDate;
import java.util.function.Supplier;

public class Main {
    public static void main(String[] args) {
        Supplier<LocalDate> currentDate = () -> {
            // LocalDate.now() gives today's date
            return LocalDate.now();
        };

        // get() executes the Supplier lambda
        LocalDate date = currentDate.get();
        System.out.println("Current date = " + date);
    }
}
```

7. Lambda expression to evaluate if a number entered is a Prime number

Code:

```
import java.util.function.Predicate;
public class Main {
    public static void main(String[] args) {
        // Predicate: input -> Integer , output -> Boolean

        Predicate<Integer> isPrime = number -> {
            if (number < 2) {
                return false;
            }
            // Check divisibility
            for (int i = 2; i <= number / 2; i++) {
                if (number % i == 0) {
                    return false;
                }
            }
            return true;
        };
        int number = 17;
        // Predicate uses test()
        boolean result = isPrime.test(number);
        System.out.println(number + " is prime: " + result);
    }
}
```

8. Lambda expression to accept 2 strings and return the concatenated value of the same

Code:

```
import java.util.function.BiFunction;
public class Main {
    public static void main(String[] args) {

        // BiFunction takes TWO inputs
        // and returns ONE output.
        // First String -> first input
        // Second String -> second input
        // Third String -> return type
        BiFunction<String, String, String> concatenate =
            (first, second) -> {
                return first + second;
            };
        // apply() executes BiFunction
        String result = concatenate.apply("Hello ", "Sahil");
        System.out.println(result);
    }
}
```

GENERATE OTP USING LAMBDA

Create a Lambda for generating a random OTP which will be formed of 5 chars, the 1st Char should be a vowel followed by 4 random numbers generated from 0-9.

Eg. A8391, U8665 and so on.

Code:

```
import java.util.function.Supplier;
public class Main {
    public static void main(String[] args) {
        Supplier<String> generateOTP = () -> {
            // Vowels from which the first character will be selected
            String vowels = "AEIOU";

            // Generate a random index from 0 to 4
            int vowelIndex = (int) (Math.random() * 5);

            char firstChar = vowels.charAt(vowelIndex);

            StringBuilder otp = new StringBuilder();

            otp.append(firstChar);

            // Generate 4 random digits from 0 to 9
            for (int i = 0; i < 4; i++) {
                int digit = (int) (Math.random() * 10);
                otp.append(digit);
            }
            // Return the final OTP
            return otp.toString();
        };

        // Generate and print OTP
        System.out.println("Generated OTP: " + generateOTP.get());
    }
}
```

Create the following using Lambda and Collection Framework:

Create a class called Transaction that contains the following data members

1. txId int
3. txAmount float
4. txStatus boolean
5. txArrears Boolean

Create 5 objects of the above Transaction object by taking input from the user and store the same in a Collection object.

Create the Lambda expressions for:

1. Getting all the Transactions from the Collection where the txAmount is > 5000.
2. Getting all the Transactions where the txStatus is false.
3. Write a Lambda Function to generate the amount due. The amount due is calculated as the txAmount + Rs. 500/- + 18% of txAmount if the txArrears is true else if txArrears if false then only the txAmount will be returned.

Code:

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.function.Function;
import java.util.function.Predicate;
class Transaction {
    int txId;
    float txAmount;
    boolean txStatus;
    Boolean txArrears;
    Transaction(int txId, float txAmount,boolean txStatus, Boolean txArrears) {
        this.txId = txId;
        this.txAmount = txAmount;
        this.txStatus = txStatus;
        this.txArrears = txArrears;
    }
    @Override
    public String toString() {
        return "Transaction{" + "txId=" + txId + ", txAmount=" + txAmount +", txStatus=" + txStatus +",
txArrears=" + txArrears +'}';
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Transaction> transactions = new ArrayList<>();
        for (int i = 0; i < 5; i++) {
            System.out.println("\nEnter Transaction " + (i + 1));

            System.out.print("Enter Transaction ID: ");
            int id = sc.nextInt();
```

```

        System.out.print("Enter Transaction Amount: ");
        float amount = sc.nextFloat();

        System.out.print("Enter Transaction Status (true/false): ");
        boolean status = sc.nextBoolean();

        System.out.print("Enter Transaction Arrears (true/false): ");
        Boolean arrears = sc.nextBoolean();

        Transaction t = new Transaction(id, amount, status, arrears);
        transactions.add(t);
    }

    System.out.println("\nTransactions where amount > 5000:");
    Predicate<Transaction> amountGreaterThan5000 = t -> t.txAmount > 5000;
    for (Transaction t : transactions) {
        if (amountGreaterThan5000.test(t)) {
            System.out.println(t);
        }
    }

    System.out.println("\nTransactions where status is false:");
    Predicate<Transaction> statusFalse = t -> t.txStatus == false;
    for (Transaction t : transactions) {
        if (statusFalse.test(t)) {
            System.out.println(t);
        }
    }

    System.out.println("\nAmount Due:");
    Function<Transaction, Float> amountDue = t -> {
        if (t.txArrears) {
            return t.txAmount + 500 + (t.txAmount * 0.18f);
        } else {
            return t.txAmount;
        }
    };

    for (Transaction t : transactions) {
        float due = amountDue.apply(t);
        System.out.println("Transaction ID: " + t.txId + " | Amount Due: " + due);
    }
}

```